Two-Sided Interval-Verified Optimality Bounds for the PGLib-OPF v23.07 Typical AC Optimal Power Flow Cases

Puyi Sun*

Computer-assisted proof candidate, not peer reviewed.
Version 1.0, 28 October 2026

Abstract

For each of the 26 typical-operating-condition (TYP) cases of the PGLib-OPF v23.07 benchmark library, with 3 to 2000 buses, we give a lower bound L and an upper bound U on the global optimum of the nonconvex AC optimal power flow problem, together with machine-checkable certificates. The upper bounds come from Krawczyk interval-Newton enclosures of exact power-flow solutions at which all operational limits are verified; the lower bounds come from sparse semidefinite and sum-of-squares dual certificates whose positive semidefiniteness and polynomial residuals are checked in interval arithmetic. All proof decisions use directed rounding on the unmodified case data. All 52 endpoints are accepted by checkers written separately from the programs that produced the certificates. Thirteen of the 26 cases have a relative gap of at most 10^{-6}; the largest instances, case1951_rte and case2000_goc, have relative gaps of about 8.1×10^{-6} and 3.8×10^{-4}. The certificates, checkers and a one-command replay script are published with this note. No claim of priority is made; the work was carried out with extensive use of AI systems, as detailed at the end.

Terminology. *Rigorous / verified* means that every numerical proof obligation (positive semidefiniteness and residual bounds for L; Krawczyk inclusion, every operational limit and the cost for U) is discharged by outward-rounded interval arithmetic on the exact decimal data of the case file. In parts of the optimal power flow literature "certified" refers to a duality gap below a solver tolerance; that weaker meaning is not used here. *Independently checked* means accepted by checker programs written separately from the certificate generators and importing nothing from them. It does *not* mean external human peer review or a formal proof in a proof assistant.

1 Problem and claim

Each case is the AC optimal power flow model of PGLib-OPF v23.07 [2] with the data of its `.m` file taken as exact decimals: complex bus voltages; nodal active and reactive power balance with shunts and π-model branches (tap ratios and phase shifts); generator active and reactive power limits; voltage magnitude limits; apparent-power limits at both ends of every rated branch; branch phase-angle-difference limits; and convex quadratic generation cost. Writing K for the feasible set and f for the cost, each case comes with a certificate pair proving

$$L \;\le\; f^* := \min_{x \in K} f(x) \;\le\; U. \tag{1}$$

Both certificates of a case record the SHA-256 digest of the same case file. Other model variants (for example current-magnitude thermal limits) and other PGLib versions are not covered.

2 Upper bounds: Krawczyk enclosures

An upper certificate stores an approximate operating point $\tilde{z}$ (voltages and free generator outputs) and a radius. The checker rebuilds the power-flow equations $F(z) = 0$ from the case file and forms the Krawczyk operator [6]

$$\mathcal{K}(\tilde{z}, Z) = \tilde{z} - C\,F(\tilde{z}) + \big(I - C\,F'(Z)\big)(Z - \tilde{z}),$$

where the preconditioner C is an approximate inverse proposed in floating point (sparse rows for large cases) and then used as exact dyadic numbers. If $\mathcal{K}(\tilde{z}, Z)$ lies in the interior of the box Z, the box contains an exact solution of $F = 0$. The checker then evaluates every operational limit and the cost over the whole box in interval arithmetic; if all limits hold, the enclosed exact solution is feasible and its cost is at most the upper end of the cost interval. Setpoints fixed in the data (equal lower and upper limits) are compared as exact decimals. For the 2000-bus case the system has 4000 real unknowns; one check took about 30–40 minutes and 160–210 MB of memory on a two-core Linux machine.

*ORCID 0009-0004-1644-5300.

3 Lower bounds: sparse dual certificates

A lower certificate is a polynomial identity in the real voltage coordinates $x = (e, f)$,

$$f(x) - t = \sum_i \sigma_i(x) + \sum_j \tau_j(x)\, g_j(x) + \sum_k \eta_k(x)\, h_k(x) + R(x), \tag{2}$$

where $g_j \geq 0$ and $h_k = 0$ are the constraints, the σ_i and the localizing multipliers τ_j are sums of squares given by Gram matrices on the cliques of a chordal extension of the network graph (order one on most cliques and order two on selected ones), the η_k are polynomial multipliers, and R is the residual. The checker proves each Gram matrix positive semidefinite after a small diagonal shift by an interval $LDL^\top$ factorization (floating point only proposes the shift), and subtracts from t the worst-case effect of all shifts and of the residual over the voltage box. Bus costs enter exactly, or through affine minorants $\lambda P + \psi(\lambda)$ whose constants are computed exactly. Thirteen cases use a chordal Shor-level (Lagrangian) certificate in a dedicated format with its own checker; the others use the sparse sum-of-squares format.

4 Results

Table 1 lists the bounds. Each value is the endpoint declared in the certificate, which the checker must prove; L is rounded down and U up to six decimals, and the relative gap $(U - L)/U$ is computed from the full-precision values. The cases with the widest gaps (case1803_snem 3.6%, case1888_rte 1.9%, case162_ieee_dtc 1.4%, case240_pserc 1.4%) are those where the order-two relaxations that fit in the available memory remain far from the optimum.

Table 1: Bounds for the 26 PGLib-OPF v23.07 TYP cases ($/h).

case	buses	L	U	$(U-L)/U$	lower certificate
case3_lmbd	3	5812.642962	5812.643005	7.33e-09	sparse SOS
case5_pjm	5	17551.890711	17551.891094	2.17e-08	sparse SOS
case14_ieee	14	2178.080425	2178.080449	1.02e-08	chordal Shor
case24_ieee_rts	24	63352.202444	63352.203674	1.94e-08	chordal Shor
case30_as	30	803.127218	803.127313	1.17e-07	chordal Shor
case30_ieee	30	8208.515455	8208.515549	1.13e-08	chordal Shor
case39_epri	39	138415.554411	138415.563671	6.69e-08	sparse SOS
case57_ieee	57	37589.336873	37589.338384	4.02e-08	sparse SOS
case60_c	60	92693.623915	92693.670117	4.98e-07	sparse SOS
case73_ieee_rts	73	189764.081348	189764.084939	1.89e-08	chordal Shor
case89_pegase	89	106968.658190	107285.675532	2.95e-03	chordal Shor
case118_ieee	118	97213.585027	97213.607809	2.34e-07	sparse SOS
case162_ieee_dtc	162	106512.858738	108075.648889	1.45e-02	sparse SOS
case179_goc	179	754128.038771	754266.421037	1.83e-04	sparse SOS
case197_snem	197	1.501257	1.501747	3.25e-04	chordal Shor
case200_activ	200	27557.570830	27557.571313	1.75e-08	chordal Shor
case240_pserc	240	3283231.188888	3329670.118665	1.39e-02	sparse SOS
case300_ieee	300	565219.433978	565220.008801	1.02e-06	sparse SOS
case500_goc	500	454945.910696	454945.987308	1.68e-07	sparse SOS
case588_sdet	588	311343.651681	313139.784598	5.74e-03	sparse SOS
case793_goc	793	258581.712068	260197.852143	6.21e-03	sparse SOS
case1354_pegase	1354	1257170.185668	1258844.004024	1.33e-03	sparse SOS
case1803_snem	1803	94825.222708	98334.547531	3.57e-02	chordal Shor
case1888_rte	1888	1375653.865114	1402531.010635	1.92e-02	sparse SOS
case1951_rte	1951	2085565.057043	2085581.868859	8.06e-06	sparse SOS
case2000_goc	2000	973063.826000	973432.478973	3.79e-04	sparse SOS

5 Independent checking and replay

Three checker programs, one for upper certificates and one for each lower-certificate format, were written separately from the generators; each reads only the raw case file and one certificate, uses the `mpmath` interval library for every decision, and uses floating point only to propose numbers that are then verified. All 52 endpoints were accepted by these checkers on a Linux machine, and again, all 52, on the author's own computer (WSL2 Ubuntu, Python 3.14) with byte-identical inputs; the smaller cases were also replayed in a separate sandbox, and many endpoints on a Windows machine. Deliberately corrupted certificates (zeroed or negated Gram blocks, negated

multipliers, altered cost pieces, declared bounds raised above the proved value, wrong case-file digest, removed equality multipliers, Krawczyk centres moved, setpoints pushed out of limits) were all rejected. The accompanying `replay.py` verifies the SHA-256 manifest of the release, checks that the declared values in its index match the certificates, and re-runs every checker.

6 Related work and position

Oustry, D'Ambrosio, Liberti and Ruiz [8] computed certified SDP lower bounds (via a Gershgorin-type eigenvalue bound on rational dual data) for AC-OPF instances with up to 6515 buses (PGLib v21.07, with current-magnitude limits); their upper bounds are local-solver (IPOPT) values without a feasibility proof. Gopinath et al. [5] closed or nearly closed many PGLib gaps with SDP-based bound tightening and valid inequalities, using floating-point solvers. Sparse moment-sum-of-squares hierarchies certify global optimality numerically on large AC-OPF instances [10, 9, 7]. Recent work strengthens relaxations with offline bound tightening [1], lifted nonlinear cuts [3] and enhanced SDP models [4]; robust AC-OPF has been solved to small solver-tolerance gaps by spatial branch and bound [11]. Interval methods for power flow feasibility are classical. We make no priority claim. What this note adds is a fixed, machine-replayable artifact in which both ends of the optimality interval are verified with directed rounding, for all 26 TYP cases up to 2000 buses, and re-checked by separately written checkers. We could not read the full text of [1]; its abstract reports numerical relaxation gaps only.

7 Limitations

1. The results rest on the correctness of the checker programs and of the interval library; separate authorship of checkers and generators reduces, but does not remove, the risk of a shared error. There has been no external review.
2. The bounds concern the PGLib v23.07 TYP model exactly as written; they say nothing about other variants, other data or real-world operation.
3. Several gaps are wide (Section 4); narrowing them needs stronger relaxations than those used here.

AI assistance and responsibility

This work was carried out with AI systems under the direction of the author, who chose the problem, set the acceptance rules, decided which results to keep, operated the machines and is responsible for this release. The certificate-generating programs were written mainly by Claude (Anthropic), with one SDP-assembly module by Codex (OpenAI); certificates were generated by Claude and, for some lower certificates, by Codex. The independent checkers were written by GPT (OpenAI) and their source was reviewed by Claude; they were run by Claude, by GPT, by Codex and on the author's computer as stated in Section 5. Mathematical review of the certificate formats was done by GPT and Claude, reviewing each other. This text was drafted by GPT and revised by Claude. *The author did not personally verify the mathematics line by line*; the claims rest on the certificates and checkers, which anyone can re-run. AI systems are not listed as authors.

Data and code

Certificates, checkers, case files and the replay script are available in the accompanying repository (https://github.com/PYCSO bounds-proofs). Code: MIT licence. Certificates and text: CC BY 4.0. Case files: PGLib-OPF v23.07 [2], CC BY 4.0, unmodified.

References

[1] X. An, C. Crozier, and S. S. Dey. Strengthening AC optimal power flow relaxation through offline bound tightening and linear approximations, 2026. arXiv:2610.32006.

[2] S. Babaeinejadsarookolaee et al. The power grid library for benchmarking AC optimal power flow algorithms, 2019. arXiv:1908.02788; data: `https://github.com/power-grid-lib/pglib-opf`, release v23.07.

[3] S. I. Bugosen, R. B. Parker, and C. Coffrin. Applications of lifted nonlinear cuts to convex relaxations of the AC power flow equations, 2024. arXiv:2404.17541.

[4] An enhanced semidefinite relaxation model combined with clique graph merging strategy for efficient AC optimal power flow solution, 2024. arXiv:2409.19609.

[5] S. Gopinath, H. L. Hijazi, T. Weisser, H. Nagarajan, M. Yetkin, K. Sundar, and R. W. Bent. Proving global optimality of ACOPF solutions. *Electric Power Systems Research*, 189:106688, 2020. arXiv:1910.03716.

[6] R. Krawczyk. Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken. *Computing*, 4:187–201, 1969.

[7] A. Le Franc et al. Minimal sparsity for second-order moment-SOS relaxations of the AC-OPF problem, 2023. arXiv:2305.19232.

[8] A. Oustry, C. D'Ambrosio, L. Liberti, and M. Ruiz. Certified and accurate SDP bounds for the ACOPF problem. *Electric Power Systems Research*, 212:108278, 2022.

[9] J. Wang, V. Magron, and J.-B. Lasserre. Certifying global optimality of AC-OPF solutions via sparse polynomial optimization, 2021. arXiv:2109.10005.

[10] J. Wang, V. Magron, J.-B. Lasserre, and N. H. A. Mai. CS-TSSOS: Correlative and term sparsity for large-scale polynomial optimization, 2020. arXiv:2005.02828.

[11] Y. Yin and V. M. Charitopoulos. Global optimization of robust AC OPF. *Systems and Control Transactions*, 5:602–609, 2026. LAPSE:2026.0277.